

Peer Review Report

Group 23 Project Evaluation: Player Transfer Prediction Using Machine Learning

Comprehensive Analytical Review

Prepared by: Group 4 Review Committee

Version: Final Draft

MIT Emergent Talent Program

Collaborative Data Science Project (CDSP)

July 2025

Abstract

This peer review evaluates the methodology, analysis, and reporting quality of Group 23's CDSP project, which applies machine learning techniques to predict the likelihood of football player transfers. The assessment is structured according to established research evaluation criteria and provides both a general review and targeted areas for improvement. Particular attention is given to model design, feature engineering, evaluation metrics, and reproducibility of results. The review aims to provide constructive, research-informed feedback to enhance the rigor and impact of the project.

Reviewers – Group 4

- Nelson Fodjo
<https://github.com/FKN237>
- Yevheniia Rudenko (Jenia)
<https://github.com/Yevheniia-Rudenko>
- Olubusayo Solola
<https://github.com/Simi-Solola>
- Khadija al Ramlawi
<https://github.com/Khadijaramlawi>
- Shaima Mohamed
<https://github.com/Shaymamohd>
- Yuri Spizhovyi
<https://github.com/yuri-spizhovyi-mit>

Table of Contents

Part I – Review

1.1 Research Question.....	5
1.2 Data Modeling Strategy.....	5
1.3 Data Analysis Strategy.....	6
1.4 Results & Interpretation.....	7
1.5 Technical & Repo Quality.....	8
1.6 Meet the Team.....	8
1.7 Summary of Review.....	8
1.8 Final Suggestions for Group 23.....	9

Part II – Areas to Improve

2.1 Clarification of Research Scope.....	10
2.2 Labeling Methodology Transparency.....	11
2.3 Feature Redundancy and Correlation Analysis.....	13
2.4 Handling of Class Imbalance.....	15
2.5 Lack of Baseline Models.....	18
2.6 Hyperparameter Tuning Omission.....	21
2.7 Cross-Validation Absence.....	23
2.8 Metric Interpretation Depth.....	25
2.9 Real-World Implications of Misclassifications.....	27
2.10 Local Model Interpretability (SHAP).....	29
2.11 Generalizability Across Leagues and Seasons.....	31

Part I – Review

1.1 Research Question

Group 23’s README states that the project aims to “build predictive models that determine the likelihood of a player being transferred based on performance data and transfer history.” This clearly indicates a machine learning focus applied to football analytics, and establishes transfer prediction as the central goal.

Strengths:

- The research topic is relevant, well-framed, and meaningful for sports analytics.
- The decision to build **position-specific models** (attackers, midfielders, defenders) reflects an understanding of domain-specific differences in player evaluation.

Areas to Improve:

- While a research aim is present, the **precise definition of the prediction target** remains vague:
 - Does “transferred” include all player movements, or just high-value transfers?
 - Is the model designed to serve scouts, analysts, or clubs?
 - What time period is being modeled (e.g., one season, multiple years)?
 - Adding a formal research question in the form of:
“Can position-specific player performance data be used to predict transfer outcomes for top-tier European footballers during the 2022–2023 season?” would help establish clarity and scope.
-

1.2 Data Modeling Strategy

Group 23 used two datasets:

1. **Players_Stats_Dataset**: player performance statistics.
2. **transfer_dataset**: information on player transfers.

In **labeling_data.ipynb**, they merged and labeled players as transferred or not, used a binary outcome (transferred = 1 or 0), and cleaned the data for modeling.

✓ Strengths:

- Good use of **domain-based feature engineering** (e.g., separate models by player position).
- Cleaning and merging steps are thorough, with good attention to duplicates, nulls, and feature reduction.

⚠ Areas to Improve:

- The labeling methodology could be documented better—how is "transferred" defined and labeled? (e.g., based on league, team, contract year?).
 - No mention of **feature correlation analysis** or **dimensionality reduction** (e.g., PCA) to avoid multicollinearity.
 - **Class imbalance** was not addressed—no evidence of balancing via SMOTE, undersampling, or weighted metrics.
-

1.3 Data Analysis Strategy

Group 23 employed Random Forest Classifiers as the primary modeling technique. Separate models were trained for each player role — **midfielders**, **attackers**, and **defenders** — allowing the model structure and feature importances to be tailored to the statistical patterns relevant for each position.

They used accuracy, precision, recall, and confusion matrices to evaluate performance.

✓ Strengths:

- Using position-specific models makes sense given how different performance metrics are by role.
- Feature importance plots are clearly generated and interpreted.
- Reasonable choices for machine learning methods (Random Forest = interpretable and handles non-linearity well).

⚠ Areas to Improve:

- No baseline model (e.g., Logistic Regression) or comparison to simpler models.
- Hyperparameter tuning is not shown or mentioned—no GridSearchCV or RandomizedSearchCV.
- No cross-validation—could lead to **overfitting** on the test set.

- Missing deeper interpretation of metrics beyond accuracy (e.g., when recall is low, what does that mean for transfers?).
-

1.4 Results & Interpretation

Each model had moderate accuracy:

- Midfielders: ~68%
- Defenders: ~63%
- Attackers: ~69%

Feature importances highlighted:

- Number of appearances
- Key passes
- Tackles (for defenders)
- Goals (for attackers)

✅ Strengths:

- Clear link between features and real-world domain knowledge (e.g., goal involvement = likely transfer).
- Insightful that **playing time metrics** are more predictive than some technical skills.

⚠️ Areas to Improve:

- Interpretation is **descriptive, but lacks depth**—what do the false positives/negatives represent in real scenarios?
- Could add **SHAP values** for richer interpretability of individual predictions.
- Unclear how generalizable the findings are (different leagues? seasons?).

1.5 Technical & Repo Quality

✓ Strengths:

- Notebook code is generally **clean, readable, and functional**.
- Modular separation by player position is logical.
- Visuals and charts are appropriate.

⚠ Issues & Suggestions:

- **Broken links** in README (technical report, non-technical report) reduce project credibility.
- Notebooks lack markdown cells summarizing findings and decisions—this reduces narrative flow.
- No deployment or reproducibility pipeline (e.g., environment files, notebook runners, Streamlit dashboard, etc.).
- The [README.md](#) lacks a compelling project summary and usage instructions.

1.6 Meet the Team

The "Meet the Team" section is present in the repository's README. It includes GitHub profiles of all group members, which adds transparency and allows attribution. However, it could be enhanced by including brief role descriptions or contributions to the project, which is considered best practice in collaborative research.

1.7 Summary of Review

Category	Rating	Comments
Research Question	✓ Good	Clear problem, but needs more explicit framing
Data Modeling	⚠ Medium	Good merging and labeling, but lacks imbalance handling
Analysis Strategy	⚠ Medium	Decent choice of algorithm but lacks tuning, validation
Results & Interpretation	⚠ Medium	Insightful features, but limited discussion of errors
Presentation & Docs	⚠ Medium	The README includes team info, but broken links and lack of project summary limit clarity.

Code Quality	✓ Good	Clean and organized, but could add markdown commentary
--------------	--------	--

1.8 Final Suggestions for Group 23

1. **Fix the broken links** (technical/non-technical report) and improve documentation.
2. Consider addressing **class imbalance** and using **cross-validation** to avoid overfitting.
3. Add a **baseline model** for comparison to validate the value of Random Forests.
4. Provide more **insightful analysis** of prediction results—e.g., who are false positives?
5. Include a clear **summary of findings** and practical recommendations in your README or final report.

Part II – Areas to Improve

2.1 Clarification of Research Scope

This issue was initially raised in Section 1.1 as part of the overall research framing. The following expands on that point by examining the definition and scope of the label in greater detail. The README does include a general project goal that touches on transfer prediction. However, the **scope and operational context remain underdefined**, which limits interpretability and reproducibility.

What's Missing:

- No definition of what constitutes a **“transfer”** (e.g., includes loans, excludes free agents?).
- No indication of the **season(s)** or timeframe covered in the analysis.
- No explanation of the **data labeling process** or target criteria (e.g., upward transfers, only high-value moves?).
- Lack of contextual framing: what leagues or competitions are included? Are lower-tier clubs part of the dataset?

Suggested Clarification:

“For the purposes of this study, we define a ‘transfer’ as any official player movement between clubs during the 2022–2023 season, as recorded in the transfer dataset. Our prediction target includes all permanent and loan moves involving clubs from Europe’s top 5 leagues.”

Providing such a statement (even briefly) would significantly improve clarity and research framing.

2.2 Labeling Methodology Transparency

The labeling methodology could be documented better — how is "transferred" defined and labeled? (e.g., based on league, team, contract year?).

What This Means

In their `labeling_data.ipynb`, Group 23 creates a binary label for each player:

- 1 if the player was transferred
- 0 if not

But they do **not clearly explain**:

- What **source** or **column** defines the transfer?
 - Is it based on:
 - Change in `club_name`?
 - A value in a `transfer_fee` column?
 - Appearance of player in both datasets with different years/clubs?
 - A condition like “**played for two clubs across seasons**”?
-

Why This Is Important

Clear labeling logic is essential because:

- It determines the **ground truth** of your model.
 - If flawed, the model learns to predict based on **ambiguous or noisy data**.
 - Without documentation, **other researchers cannot reproduce** or trust your results.
-

Example of What's Missing

They could include markdown or a README snippet like:

_“To label players as ‘transferred’, we merged the transfer dataset with player stats and created a binary label:

- **1** if a player appeared in the transfer dataset for the same season as their performance stats
- **0** otherwise.
We only considered official transfers during the 2022–2023 season. Players switching clubs between major leagues were prioritized.”_

Or in notebook form:

Labeling Methodology

We labeled each player as `1` (Transferred) if their name appeared in the `transfer\_dataset` for the target season. Players not listed in that dataset were assigned a `0`. We did not filter by transfer fee or club tier. Loans and free transfers were included as transfers.

How They Can Improve

1. Add a **clear markdown explanation** in `labeling_data.ipynb`.
2. Mention:
 - Columns used (`player_name`, `season`, `team`, etc.)
 - Handling of **loans**, **expired contracts**, **free agents**
 - Whether **league tiers** or **transfer fees** were considered
3. Provide **one example row** of a labeled transfer vs non-transfer.

2.3 Feature Redundancy and Correlation Analysis

No mention of feature correlation analysis or dimensionality reduction (e.g., PCA) to avoid multicollinearity.

What This Means

In the uploaded notebooks ([rf_predicting_transfer_.ipynb](#)), Group 23 trains Random Forest models using many numerical features from the player stats dataset.

However, they **don't show any analysis of**:

- **Correlation between features** (e.g., heatmap, correlation matrix)
 - **Redundant or highly collinear features** (e.g., total passes vs. accurate passes)
 - **Techniques like PCA or feature selection** to reduce overlap
-

Why It Matters — Especially in Sports Data

In football stats:

- Features like [passes_total](#), [passes_accurate](#), and [key_passes](#) are likely highly correlated.
 - Random Forests are somewhat robust to multicollinearity, but **redundant features can still confuse feature importance** and lead to **overfitting** or inflated model complexity.
 - More importantly, **interpretability suffers** — the model might favor features that are essentially “saying the same thing.”
-

Example of What's Missing

They could have added a code cell like this:

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
corr_matrix = df.corr()
plt.figure(figsize=(12,10))
sns.heatmap(corr_matrix, annot=False, cmap='coolwarm', fmt='.2f')
plt.title("Feature Correlation Heatmap")
```

And then removed or merged features with correlation > 0.9.

Or run a **Variance Inflation Factor (VIF)** test:

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif = pd.DataFrame()
vif["feature"] = X.columns
vif["VIF"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
```

✓ Suggested Improvements

1. **Run correlation matrix** before modeling.
2. **Drop or combine highly correlated features** (e.g., total vs accurate metrics).
3. **(Optional) Apply Principal Component Analysis (PCA)** for feature compression — useful if they want to deploy the model.
4. Clearly explain these decisions in the notebook or README.

📌 Suggested Markdown Block for Their Notebook

markdown

Feature Correlation Analysis

Before training, we computed a correlation matrix to identify multicollinearity among features. Features with correlation > 0.9 were candidates for removal or merging (e.g., total vs. successful passes). This step helps prevent overfitting and ensures clearer feature importance interpretation.

2.4 Handling of Class Imbalance

Class imbalance was not addressed — no evidence of balancing via SMOTE, undersampling, or weighted metrics.

What This Means

In Group 23's notebooks for predicting transfers (attackers, midfielders, defenders), they used a **binary target**:

- **1** = Transferred
- **0** = Not Transferred

But they **never explore or visualize the class distribution**, nor apply any techniques to deal with it.

Why This Matters

In real football datasets, **the majority of players are *not* transferred** in a given window — meaning:

- The dataset is likely **imbalanced** (e.g., 80% not transferred vs 20% transferred).
- A model can achieve **high accuracy just by predicting “no transfer”** all the time.
- Metrics like **accuracy** become misleading.

For example, a model with 90% accuracy might still miss most actual transfers — which defeats the purpose.

What They Could Have Done

Check class balance at the start:

```
python
```

```
y.value_counts(normalize=True).plot(kind='bar', title='Class Distribution')
```

1. Apply one or more **balancing techniques**:

- **SMOTE** (Synthetic Minority Oversampling Technique) to create synthetic samples of minority class.
- **Undersampling** the majority class.
- Use **class_weight="balanced"** in the RandomForestClassifier.
- **Evaluate with precision, recall, F1-score, ROC AUC**, not just accuracy.

✓ Suggested Improvement Steps

1. Add a markdown explanation of **class imbalance risks**.

Re-train the models using:

python

```
RandomForestClassifier(class_weight='balanced', random_state=42)
```

2.

Re-evaluate with:

python

```
from sklearn.metrics import classification_report  
print(classification_report(y_test, y_pred))
```

3.

Optionally try SMOTE:

python

```
from imblearn.over_sampling import SMOTE
sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X_train, y_train)
```

4.

 Suggested Markdown Block for Notebook

markdown

Addressing Class Imbalance

The dataset had an imbalanced label distribution (e.g., 22% transferred vs. 78% not transferred). To address this, we applied class weighting in the Random Forest model and evaluated using recall and F1-score rather than accuracy alone. This ensures the model is learning to identify actual transfers and not just predicting the majority class.

2.5 Lack of Baseline Models

No baseline model (e.g., Logistic Regression) or comparison to simpler models.

What This Means

Group 23 went straight to using **Random Forest Classifiers**, which are:

- More powerful than linear models
- But also more complex, harder to interpret
- Prone to overfitting, especially without hyperparameter tuning or balancing

Yet they **didn't establish a baseline** using a simpler model like:

- **Logistic Regression**
 - **Naive Bayes**
 - Even “**predict all 0s**” **dummy model** for reference
-

Why It Matters

Without a baseline:

- You can't **quantify how much better** your advanced model is.
- You might overestimate model performance (“68% accuracy” sounds good, until you realize a dumb model gives 65%).

A **Logistic Regression baseline** provides:

- A quick benchmark using linear decision boundaries
- Easily interpretable coefficients

- A sanity check for your data pipeline

Example

They could have added:

python

```
from sklearn.linear_model import LogisticRegression
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_lr))
```

Then compared **precision/recall/F1-score** with the Random Forest model:

- If the RF is only *marginally better*, maybe it's not worth the extra complexity.

Suggested Improvement

1. Train at least one **baseline model**:
 - Logistic Regression for tabular data
 - DummyClassifier (predicting majority class)
2. Include a **comparison table** of key metrics:

Model	Accuracy	Precision	Recall	F1
-------	----------	-----------	--------	----

Logistic Regression	63%	55%	60%	57%
Random Forest	68%	62%	64%	63%

3. Interpret what the gain **really means**.

 Suggested Markdown Block

markdown

Baseline Comparison

We trained a Logistic Regression model to serve as a baseline. While it performed slightly worse than our Random Forest model, it confirmed that more advanced techniques were justified. The improvement in recall and F1-score supports using Random Forests for this classification task.

2.6 Hyperparameter Tuning Omission

Hyperparameter tuning is not shown or mentioned — no **GridSearchCV** or **RandomizedSearchCV**.

What This Means

Group 23 trained Random Forest models with default parameters:

python

```
model = RandomForestClassifier(random_state=42)
```

This is **perfectly fine as a starting point**, but:

- Random Forests have many tunable parameters (e.g., **n_estimators**, **max_depth**, **min_samples_split**, **max_features**, etc.)
 - **Default settings may not be optimal** for their specific data
 - There's **no attempt to test alternate settings** using standard hyperparameter tuning techniques
-

Why It's a Problem

Without tuning:

- The model might **underfit** (too shallow, too few trees)
 - Or **overfit** (too deep, too many trees)
 - You may be missing out on **better generalization performance**
-

What They Could Have Done

Use **GridSearchCV** to search exhaustively:

python

```

from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'max_features': ['sqrt', 'log2'],
    'min_samples_split': [2, 5, 10]
}

grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5, scoring='f1')
grid.fit(X_train, y_train)

print("Best Parameters:", grid.best_params_)

```

Or use [RandomizedSearchCV](#) for a quicker sampling-based approach.

Then evaluate and report improvements.

✓ Suggested Improvements

1. Include a **hyperparameter tuning section** in each model notebook.
 2. Use either [GridSearchCV](#) or [RandomizedSearchCV](#).
 3. Report the **best parameters** and whether they meaningfully improved performance.
 4. Log training time if relevant (to show cost-benefit tradeoff).
-

📌 Suggested Markdown Block

markdown

Hyperparameter Tuning

We used `GridSearchCV` to identify optimal hyperparameters for the Random Forest models. This helped reduce overfitting and improved precision and F1-score slightly compared to default settings. For example, increasing `max_depth` and tuning `min_samples_split` led to more stable predictions across folds.

2.7 Cross-Validation Absence

No cross-validation — could lead to overfitting on the test set.

What This Means

In their Random Forest modeling notebooks ([rf_predicting_transfer*.ipynb](#)), Group 23 follows this typical sequence:

1. Split data into `X_train` and `X_test`
2. Train the model on the training set
3. Evaluate on the test set using `accuracy_score`, confusion matrix, etc.

However, they do **not use k-fold cross-validation** or any **resampling method** to validate performance.

Why It's a Problem

Evaluating a model on a single train-test split:

- **Can lead to overfitting** to that specific test set
 - Gives **unstable performance estimates**, especially with smaller datasets
 - Doesn't show how the model would perform on **unseen samples in general**
-

What They Could Have Done Instead

Use **k-fold cross-validation**:

python

```
from sklearn.model_selection import cross_val_score

model = RandomForestClassifier(random_state=42)
```

```
scores = cross_val_score(model, X, y, cv=5, scoring='f1')
print("Cross-validated F1 scores:", scores)
print("Average F1:", scores.mean())
```

Or for more insight:

python

```
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5)
for train_idx, test_idx in skf.split(X, y):
    ...
```

This provides:

- A **distribution of scores** across folds
- Better insight into **model variance**
- Reduces chance of lucky/unlucky test split

Suggested Improvement

1. Add **5-fold or 10-fold cross-validation** to model evaluation.
Report mean + std of metrics like F1 or ROC-AUC.
2. Compare cross-validation performance to single test split.

Suggested Markdown Block

markdown

Model Validation with Cross-Validation

To ensure our model generalizes well and isn't overfitting to a specific test set, we used 5-fold cross-validation on the full dataset. This allowed us to report more robust F1-scores and precision-recall estimates, and revealed consistent performance across different data splits.

2.8 Metric Interpretation Depth

Missing deeper interpretation of metrics beyond accuracy (e.g., when recall is low, what does that mean for transfers?).

What This Means

In their notebooks, Group 23 mostly reports:

- **Accuracy**
- Confusion matrix
- Sometimes precision and recall (but not discussed)

However, they do **not explain what those metrics mean** in the context of predicting player transfers, especially:

- What does a **low recall** or **low precision** imply for users of this model?
 - What are the **consequences of false positives** or **false negatives**?
-

Why It Matters

Accuracy can be misleading in imbalanced datasets (e.g., if 80% of players are not transferred, a model can achieve 80% accuracy by always predicting "no transfer").

Instead, real insight comes from:

- **Precision:** Of all predicted transfers, how many were correct?
 - **Recall:** Of all actual transfers, how many did we catch?
-

Example Interpretation

- A **low recall** (e.g., 40%) means:

“We’re missing 60% of players who actually got transferred. This might make the model unreliable for talent scouts who want to identify all possible transfer candidates.”

- A **low precision** (e.g., 30%) means:

“Most of the players we flag as likely to transfer are actually not. This could waste resources or lead to scouting irrelevant players.”

- **F1-score** balances both, and should be preferred when both false positives and false negatives are costly.

✓ Suggested Improvements

1. Include **classification_report** output and interpret each metric.
2. Discuss **which error is worse** in this domain:
 - False positives (predicting a transfer that won't happen)
 - False negatives (missing a real transfer)
3. Possibly explore **ROC-AUC** or **PR AUC** curves to analyze threshold behavior.

📌 Suggested Markdown Block

markdown

Metric Interpretation

While our model achieved 69% accuracy, we observed a recall of 41%, meaning it missed nearly 60% of actual transfers. In a real-world scouting or contract-planning scenario, this could lead to significant blind spots. Improving recall would allow the model to catch more genuine transfer cases, even at the expense of more false positives.

Precision was 58%, indicating that about half of our flagged transfer candidates were truly transferred. A higher precision would reduce false alarms, which is desirable if resources for follow-up scouting are limited.

2.9 Real-World Implications of Misclassifications

Interpretation is descriptive, but lacks depth — what do the false positives/negatives represent in real scenarios?

What This Means

Group 23 does a good job presenting:

- Which features are important
- How well the model performs numerically

But they **stop short of exploring the practical implications** of misclassifications:

- What does it mean if the model **falsely predicts** a player will be transferred (false positive)?
- What does it mean if it **misses** a player who *was* transferred (false negative)?
- How could these errors **affect stakeholders** (e.g., scouts, clubs, agents)?

Real-World Examples:

Error Type	What It Means	Real-World Impact
False Positive	Player predicted to transfer but didn't	Waste of resources scouting or negotiating
False Negative	Player not predicted to transfer but did	Missed opportunity to prepare or bid on the player

Suggested Improvements

1. In each model notebook, show **examples of false positives and false negatives**:
 - Use `.loc[]` with indexes from the confusion matrix to pull rows

Example:

python

```
X_test[(y_test == 1) & (y_pred == 0)] # False negatives
X_test[(y_test == 0) & (y_pred == 1)] # False positives
```

-
- 2. Write a few bullet points in markdown like:

“False positives often include players with high key passes but low actual play time — perhaps indicating overestimated potential by the model.”
- 3. Discuss what **stakeholders** might prefer:
 - Talent scouts may tolerate false positives if it means catching rising stars (favor recall).
 - Club managers may want to avoid false alarms to save budget (favor precision).

Suggested Markdown Block

Interpreting Prediction Errors

False Positives:

Our model flagged several players as likely transfer candidates who were not transferred. Upon review, many had standout performance stats but limited game time -- suggesting overestimation based on limited appearances.

False Negatives:

We missed several actual transfers, particularly among younger players with modest performance stats. These players may have been transferred based on potential or strategic decisions not captured by the data.

Implication:

Depending on the stakeholder, the model may need tuning toward either higher recall (to catch more true transfers) or higher precision (to reduce false alarms). For example, a club preparing to replace aging midfielders may prefer higher recall to explore more options.

2.10 Local Model Interpretability (SHAP)

Could add SHAP values for richer interpretability of individual predictions.

What This Means

Group 23 uses **feature importance from Random Forests**, which shows:

- On average, which features matter most in making predictions.

However, this is a **global explanation** — it tells us what the model cares about overall.

What's missing is **local interpretability**:

- For a **single player**, why did the model predict transfer = 1?
- Was it due to high passes? Age? Club history?

This is where **SHAP (SHapley Additive exPlanations)** shines.

Why SHAP Is Valuable

SHAP values explain the **contribution of each feature** to a specific prediction:

- Shows whether each feature **pushed the prediction up or down**
 - Gives **per-player breakdowns**
 - Helps understand and debug model behavior
 - Improves **trust and transparency**
-

Example Use

They could add this to a notebook:

python

```
import shap
```

```
# Fit model
```

```
model.fit(X_train, y_train)

# Initialize SHAP explainer
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Visualize for one player
shap.initjs()
shap.force_plot(explainer.expected_value[1], shap_values[1][0], X_test.iloc[0])
```

Or show a **summary plot**:

python

```
shap.summary_plot(shap_values[1], X_test)
```

Suggested Improvement

1. Use SHAP to explain 1–3 **individual player predictions** (true positive, false positive, false negative).
2. Add summary plot for model interpretability.
3. Use plots to discuss **why the model made specific calls**, especially borderline cases.

Suggested Markdown Block

markdown

Explaining Predictions with SHAP

To better understand why the model predicted a transfer for individual players, we used SHAP values. These reveal the contribution of each feature to a specific prediction. For example, one midfielder was flagged due to a combination of high passing accuracy, frequent key passes, and age below 24 -- even though their goal tally was low.

This level of explainability can help club analysts or recruiters trust the model's suggestions, and dig into "why" certain predictions were made.

2.11 Generalizability Across Leagues and Seasons

Unclear how generalizable the findings are (different leagues? seasons?).

What This Means

Group 23's project builds prediction models for player transfers, but:

- They do not clearly specify the **time range** of data used (e.g., only 2022–2023 season? multiple years?).
- There's no mention of **league coverage** — are these players all from top-tier European leagues? Mixed leagues? Lower divisions?
- No analysis is done on **how stable their findings would be across time or regions**.

So while the model may work for *this dataset*, it's not clear:

- Whether it would still work for **future seasons**
 - Or for **other leagues** (e.g., MLS, Ligue 1, or Saudi Pro League)
-

Why It Matters

- A model trained on **one league or one season** may learn **league-specific transfer patterns**.
- For example, transfer dynamics in the **Premier League** are very different from those in **Serie B**.
- If the dataset lacks **temporal or geographic diversity**, the model may fail when deployed elsewhere.

This affects:

- **Reproducibility**
 - **Transferability of findings**
 - **Confidence in applying the model beyond the training data**
-

Suggested Improvements

1. Explicitly state:
 - What seasons are covered?
 - What leagues or countries are represented?
 - How many players per league?
 2. Add a discussion on **limitations of the dataset** and potential **biases**:
 - Example: “Most data points come from top 5 European leagues — findings may not hold for smaller leagues.”
 3. Consider evaluating performance **by subgroup**:
 - Accuracy by league
 - Accuracy by season (if multi-year data)
 4. Mention **future validation**:
 - “We plan to test on next season’s data or a holdout league.”
-

 Suggested Markdown Block

Generalizability and Limitations

Our dataset primarily contains player data from the 2022-2023 season and focuses on top-tier European leagues (e.g., EPL, La Liga, Bundesliga). As a result, our model may capture transfer dynamics unique to these leagues.

We acknowledge this limitation and recognize that applying the model to players in lower leagues or different regions (e.g., MLS, South American leagues) may result in reduced accuracy. Future work should include multi-season data and diverse leagues to test temporal and geographic generalizability.
